

同濟大學

TONGJI UNIVERSITY

《计算机系统实验》

实验报告

实验名称

μ C/OS-II 系统移植

实验成员

曾崇然

日期

二零二四年 5 月 29 日

1、实验目的

- 1) 通过修改、编译、移植 $\mu\text{C}/\text{OS-II}$ 嵌入式操作系统到 NEXYS 4 DDR 开发板上，回顾和复习编译原理、操作系统等课程的基本知识。
- 2) 通过整个实验过程，将计算机组成原理，编译原理，操作系统等计算机相关知识融会贯通。

2、实验内容

按照《自己动手写 CPU》思路，增加 Wishbone 总线，GPIO、UART、Flash 控制器、SDRAM 控制器。利用 Ubuntu 上建立交叉编译环境对 $\mu\text{C}/\text{OS-II}$ 系统进行改写、编译。最终成功将 $\mu\text{C}/\text{OS-II}$ 系统移植到 Digilent NEXYS 4 DDR 开发板上并验证。

3、实验环境

集成开发环境：Vivado 2019.1

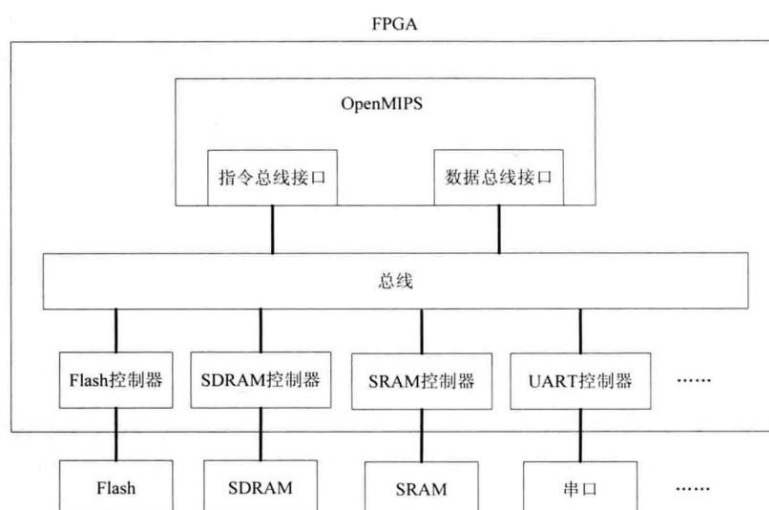
开发平台：NEXYS 4 DDR Atrix-7

4、实验步骤

4.1.修改 89 条 MIPS CPU

在实验一 CPU 改造实验中完成了 89 条 MIPS CPU 的实现，实现了 CP0 协处理器、异常处理等功能。

为了方便的接入新设备而不用频繁的修改 CPU 的接口，可以将 CPU 通过总线接口模块挂载到总线上，各种设备也挂载到总线上，本次实验的总体框架如图：



具体步骤为：

①修改处理器接口，将指令存储器和数据存储器的接口替换为指令 WishBone 总线接口和数据 Wishbone 接口。

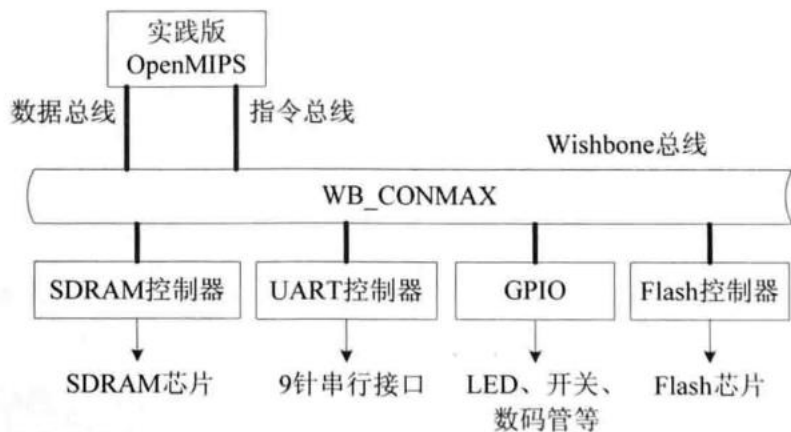
②实现 Wishbone 总线接口模块，将处理器对数据和指令的访问信号转化为标准的 Wishbone 总线接口信号，模块定义在 wishbone_bus_if.v 文件中。

③修改处理器内部的一些模块和连接：PC 不再直接从指令存储器获取指令，而是经过总线接口模块；MEM 木块不再直接访问数据存储器，而是通过 Wishbone 总线接口模块转化为 Wishbone 总线接口信号；由于从 flash 和 SDRAM 获取指令和数据，导致访问时间多余一个周期，所以修改取指和访存阶段，以及控制模块，以使得在数据和指令没有取到的时候请

求流水线的暂停。

4.2.为 CPU 添加 Wishbone 总线

本次实验 Wishbone 总线使用的是 OpenCores 站点提供的开源项目 WB_CONMAX，这是一个 Wishbone 总线互联矩阵，采用的是交叉互联方式，允许多对主从设备同时进行通信。



具体模块如下

wb_conmax_arb.v	2024/5/27 13:55	V 文件
wb_conmax_defines.v	2024/5/27 13:55	V 文件
wb_conmax_master_if.v	2024/5/27 13:55	V 文件
wb_conmax_msel.v	2024/5/27 13:55	V 文件
wb_conmax_pri_dec.v	2024/5/27 13:55	V 文件
wb_conmax_pri_enc.v	2024/5/27 13:55	V 文件
wb_conmax_rf.v	2024/5/27 13:55	V 文件
wb_conmax_slave_if.v	2024/5/27 13:55	V 文件
wb_conmax_top.v	2024/5/27 13:55	V 文件

4.3.添加 GPIO 模块

GPIO(General Purposr Input Output)是以位为单位进行数字输入输出的 I/O 接口，作为单纯的通用输入/输出 I/O。

本次实验直接使用 OpenCores 站点提供的开源项目 GPIO IP Core，具体模块如下：

gpio_defines.v	2024/5/27 13:55	V 文件
gpio_top.v	2024/5/27 13:55	V 文件

4.4.增加 UART 控制器

UART 即通用异步收发器，是广泛使用的串行数据传输协议。

本次实验采用的 UART 控制器是 OpenCores 站点提供的开源项目 UART16550 IP Core，具体模块如下：

raminfra.v	2024/5/27 13:55	V 文件	6 KB
timescale.v	2024/5/27 13:55	V 文件	1 KB
uart_debug_if.v	2024/5/27 13:55	V 文件	6 KB
uart_defines.v	2024/5/27 13:55	V 文件	11 KB
uart_receiver.v	2024/5/27 13:55	V 文件	18 KB
uart_regs.v	2024/5/27 13:55	V 文件	30 KB
uart_rfifo.v	2024/5/27 13:55	V 文件	12 KB
uart_sync_flops.v	2024/5/27 13:55	V 文件	6 KB
uart_tfifo.v	2024/5/27 13:55	V 文件	9 KB
uart_top.v	2024/5/27 13:55	V 文件	12 KB
uart_transmitter.v	2024/5/27 13:55	V 文件	13 KB
uart_wb.v	2024/5/27 13:55	V 文件	5 KB

4.5.增加 Flash 控制器

由于 NEXYS 4 DDR Atrix-7 的板载 flash 采用 SPI 通信协议，因此《自己动手写 CPU 上的 SPI 控制器》不能够直接进行使用，需要重新根据 SPI 协议编写 flash 控制器，模块定义在 flash_rom.v 文件中。

其接口定义如下：

```
module flash_rom(
    input wire wb_clk_i, // Wishbone总线的时钟信号
    input wire wb_rst_i, // 复位信号，用于初始化或重置模块状态
    input wire wb_cyc_i, // 表示一个Wishbone总线周期的开始和结束
    input wire wb_stb_i, // 选通信号，表明Wishbone主设备发起了一个有效的传输
    input wire wb_we_i, // 写使能信号，区分读写操作
    input wire [3:0] wb_sel_i, // 字节选择信号，用于选择32位数据中的有效字节
    input wire [23:0] wb_adr_i, // 地址信号，指定访问flash存储的地址
    input wire [31:0] wb_dat_i, // 写数据信号，从Wishbone总线传输到flash存储的数据
    output reg [31:0] wb_dat_o, // 读数据信号，从flash传输到Wishbone总线的数据
    output reg wb_ack_o, // 应答信号，表示数据传输完成

    input wire flash_continue, // 外部信号，控制模块的持续操作

    output reg cs_n, // 片选信号，控制flash存储设备的使能状态
    input sdi, // 串行数据输入
    output reg sdo, // 串行数据输出
    output reg wp_n, // 写保护信号
    output reg hld_n // 保持信号
);
```

该模块主要功能包括：

Wishbone 总线接口：与 FPGA 其他部分通信，支持读写命令的传递。

SPI Flash 操作：通过 SPI 协议控制 Flash 的读写操作。

状态机控制：利用多个状态来管理 Flash 操作的不同阶段，如发送指令、地址，以及读写数据。

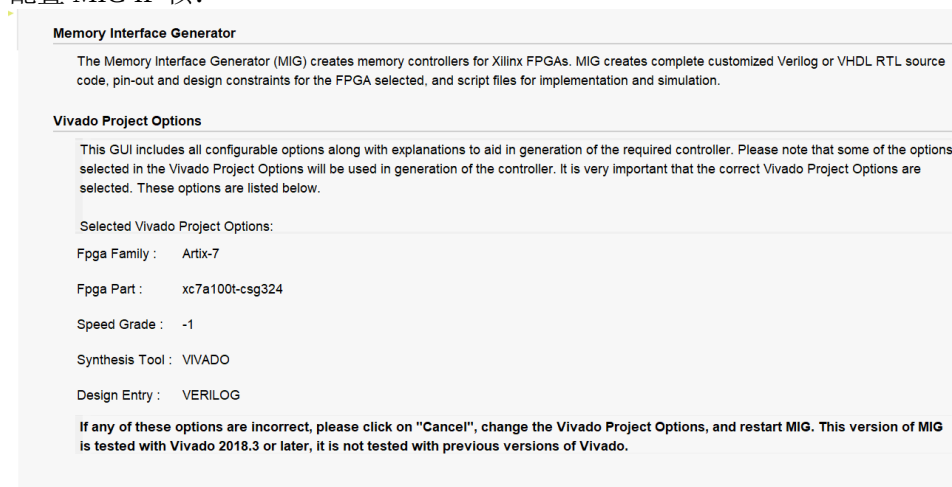
数据缓存和处理：暂存要写入或已读取的数据，以及生成和接收 SPI 数据流。

信号生成：生成必要的控制信号如片选、时钟信号等，确保 SPI 协议的正确执行。

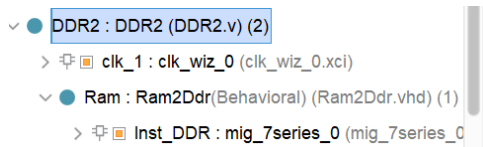
4.6.增加 SDRAM 控制器

SDRAM 是同步动态随机访问存储器，于《自己动手写 CPU》书上使用的 SDRAM 控制器模块不同，本次实验使用的是 NEXY4 DDR 板载的 DDR2 模块。本次实验使用 Xilinx 提供的简化控制 DDR 的内存控制器 IP 核，Memory Interface Generator(MIG)。

配置 MIG IP 核：



IP 核生成后，将 DDR 物理接口添加到顶层模块，控制器代码采用 Digilent Inc 公司的开源代码，DDR 控制器结构如下：



4.7.编写顶层模块

编写顶层模块，实例化各个模块，并将处理器和外部设备连接起来，模块定义在 openmips_min_sopc.v 文件中。

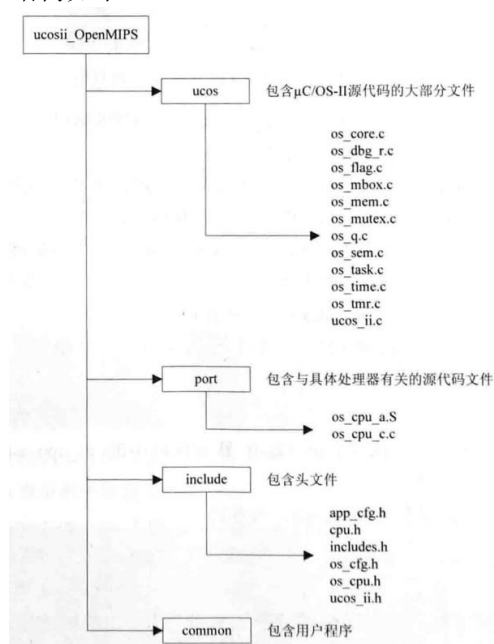
4.8.修改 μ C/OS-II 操作系统

根据《自己动手写 CPU》步骤，基于 MIPS M14K 的移植代码，对 ucousii 嵌入式操作系统做如下的修改：

1) 构建项目目录：

include	2024/5/30 10:47	文件夹
port	2024/5/30 10:46	文件夹
ucos	2024/5/30 10:44	文件夹
common	2024/5/30 10:52	文件夹

结构如下：



2) 修改 os_cpu.h 文件

3) 修改 os_cpu_a.S 文件

4) 修改 os_cpu_c.c 文件

4.9.编写测试程序

创建测试程序，包含两个文件 openmips.h 和 openmip.c，分别放于 include 和 common 文件夹下。该程序实现的功能是通过 UART 输出指定字符串中的一个字符，然后延迟大概 100ms，再输出下一个字符。

4.10.在 Ubuntu 环境下编译操作系统，得到 bin 文件

在每个文件夹下面编写对应的 Makefile 文件，进行编译，获得 ucousii.bin 文件，再使用 MergeBin 将 ucousii.bin 和 BootLoader.bin 合并成一个二进制文件 OS.bin，这样当处理器运行时，会首先运行 BootLoader，将代码复制到 SDRAM 中，然后跳转到 SDRAM 中，开始运行 μ C/OS-II。

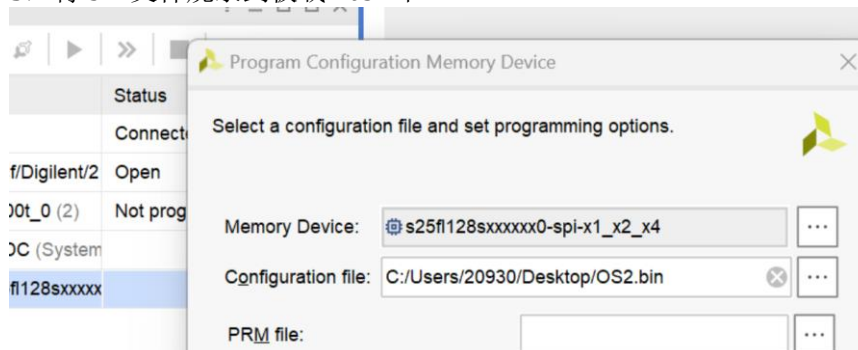
4.11.下板验证

1) 编写 xdc 文件，将接口配置到板载的开关 (gpio_i)，七段数码管 (gpio_o)，按钮 (rst_n

和 gpio_i), USB (uart_in, uart_out), Flash (sdo, sdi, wp_n, hld_n, cs_n)。

2) 综合、布线、生成 bit 流

3) 将 bin 文件烧录到板载 flash 中

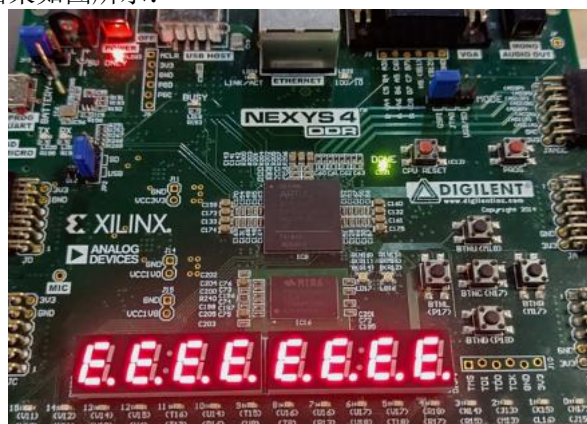


4) 将生成的 bit 流下载到开发板上

5、实验结果

5.1.gpio 和 flash 测试

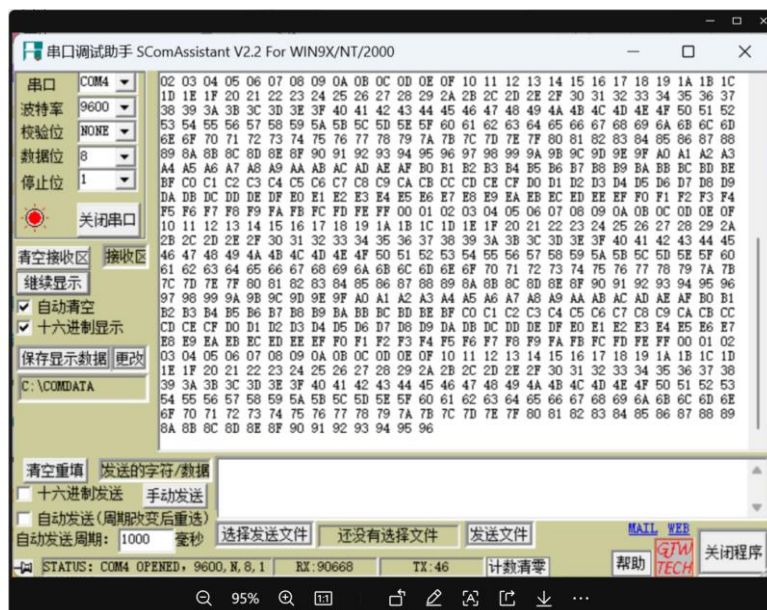
使用《自己动手写 CPU》上的单元测试程序，编译得到 bin 文件，烧录到 flash 中，运行结果如图所示：



结果解释：由于 DE2 平台的七段数码管的每一段都可以接上 GPIO，所以能使四个数码管直接显示不同的数字，而 NEXY4 平台上的数码管提供的引脚只有 8 个，仅用于控制八个数码管的亮灭以及每个数码管的相同位置，因此此处我配置引脚使其只显示最后一个字母，即大写的 E。实验结果符合预期，表明 gpio 模块和 flash 控制器正确。

5.2.uart 测试

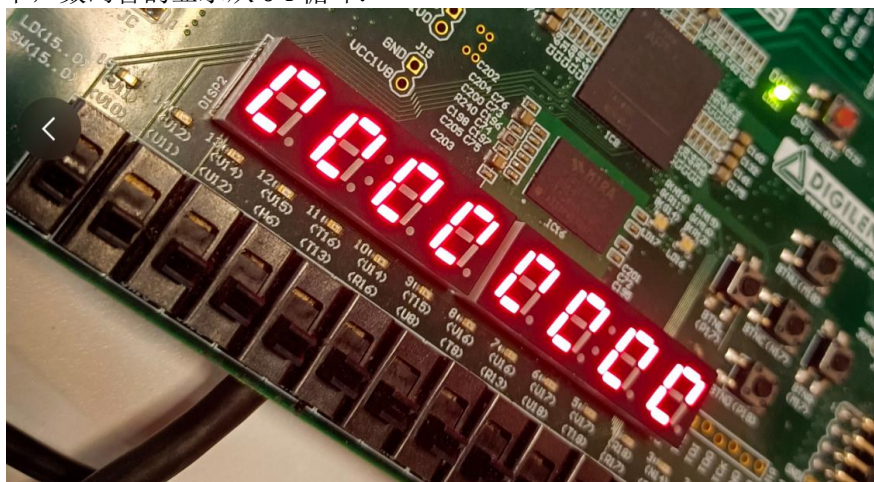
自己动手写 CPU 使用的开发平台的时钟频率为 27M，而 NEXY4 平台上是 100M，为了能使波特率为 9600，经计算后修改分频系数的高位为 02，低位为 8B，编译测试程序形成 bin 文件，烧录到 flash 中，打开串口调试工具，结果如图：



结果说明：发送的信号主次加一，然后循环，符合预期，说明 uart 模块的配置成功。

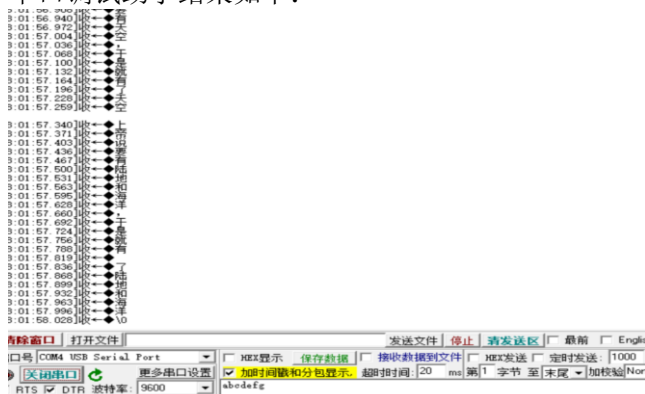
5.3.操作系统移植

将与 BootLoader.bin 连接形成的 OS.bin 烧录到 flash 中，拨动拨码开关，七段数码管显示如下，数码管的显示从 0-f 循环：



gpio 的输出是用户程序中的 count 的值，count 每 10 次时钟中断增加一次，显示也增加，不过由于此处只显示低 4 位，因此表现为低位循环，这说明用户程序正确运行，操作系统移植成功。

串口调试助手结果如下：



PC 成功接收到了来自串口的信息，说明程序正确运行且串口功能完成。

6、实验总结

6.1.问题

1) flash 控制器编写的问题：由于板载 flash 型号和《自己动手写 CPU》上差别很大，通信协议也不相同，因此在完成此控制模块时遇到了很大的困难，在 github 上学习了开源的控制模块，经过阅读指导书，查阅资料，理解其协议和时序逻辑，完成了板载 flash 的控制。

2) SDRAM 控制器的编写：IP 核的配置很复杂，且控制模块的编写毫无思路。使用了 Diligent Inc 公司开源的控制模块，和 vivado 提供的简化的 mig IP 核完成了板载 SDRAM 的控制。

3) UART 波特率的问题，最开始串口收到的信息一直是乱码，经检查发现是因为《自己动手写 CPU》中配置的时钟频率为 27M，与我们的开发平台不同，经计算和修改后，成功完成了串口通信的任务。

4) 七段数码管的配置，七段数码管开始的输出一直很混乱，研究后发现是因为我们的七段数码管控制方式与书上不同，并不是 GPIO 输出本身存在问题，经修改连接后输出了符合预期的值。

5) 操作系统的修改问题：跟着《自己动手写 CPU》进行操作系统的修改和移植，没遇到很大的问题，只是用户程序中的时钟频率需要额外的修改。

6) 操作系统的编译问题：我电脑上装载的是 64 位 Ubuntu，使用不了提供的 32 位的 gnu 工具链，因此自己安装了 64 位的工具链进行编译，增加了修改 Makfile 的工作量，同时由于只给出了 32 位的 BenMerge.exe 而未给出 c 源文件，导致无法进行最后的 bin 合并工作，最后是在同学的电脑上完成了操作系统的编译工作。

6.2 心得体会

在本次实验中，根据指导书完成了实践版的处理器，将其接到了总线上并使用总线连接了外部的设备，并移植了嵌入式操作系统到该处理器上。

在这个过程中，我对计算机组成原理，其基本结构，运行方式，已经各个设备之间的通信有了更加深刻的理解，同时通过对操作系统的修改和编译，也让我更加深入的了解操作系统的相关知识和编译的基本过程和作用。

经过本次实验，感觉自己对计算机的基本核心课程有所贯通，将知识统一了起来。